

WEEK 10

Task 1

Skill objective:

To Parse Combined Redirection, Duplicate File Descriptors, Merge Streams, Verify Ordering, Handle Edge Cases, Test Integrated Outputs.

Implementation / work:

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {    const char *combined_file = "merged_output.log";    printf("[Task 1]
Parsing Combined Redirection: 'cmd > %s 2>&1'\n", combined_file);

    pid_t pid = fork();    if (pid == 0) {        int fd =
open(combined_file, O_WRONLY | O_CREAT | O_TRUNC, 0644);        if (fd <
0) { perror("open"); exit(1); }

        /* Step 1: stdout redirected to file */        dup2(fd, STDOUT_FILENO);
/* Step 2: stderr duplicated to wherever stdout is currently pointing */
dup2(STDOUT_FILENO, STDERR_FILENO);        close(fd);

        /* Write to both stdout and stderr */        fprintf(stdout, "[STDOUT STREAM]
Standard informational report message.\n");        fprintf(stderr, "[STDERR STREAM]
Standard error diagnostic alert message.\n");        exit(0);    }    waitpid(pid,
NULL, 0);

    printf("[Task 1] Verifying unified merged stream file '%s':\n", combined_file);
system("cat merged_output.log");    return 0; }
```

Output:

```
charanteja_a@charan:~$ nano exp10s.c
charanteja_a@charan:~$ gcc exp10s.c -o exp10s
charanteja_a@charan:~$ ./exp10s
[Task 1] Parsing Combined Redirection: 'cmd > merged_output.log 2>&1'
[Task 1] Verifying unified merged stream file 'merged_output.log':
[STDOUT STREAM] Standard informational report message.
[STDERR STREAM] Standard error diagnostic alert message.
charanteja_a@charan:~$ |
```

Task 2

To Parse Complex Commands, Combine Pipes and Redirections, Resolve Execution Order, Validate Syntax, Generate Execution Plans, Test Scenarios.

Implementation / work:

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {    /* Setup input dataset */    FILE *f =
fopen("raw_metrics.txt", "w");    fprintf(f, "apple 100\nbanana 200\ncherry
150\napple 300\nbanana 400\n");    fclose(f);

    printf("[Task 2] Complex Command: 'grep apple < raw_metrics.txt | sort > filtered.txt'\n");
int pipefd[2];    pipe(pipefd);

    if (fork() == 0) {        /* Stage 1: input
redirection + pipe write */        int in_fd =
```

Skill objective:

```

open("raw_metrics.txt", O_RDONLY);          dup2(in_fd,
STDIN_FILENO);          close(in_fd);

    dup2(pipefd[1], STDOUT_FILENO);
close(pipefd[0]); close(pipefd[1]);
execlp("grep", "grep", "apple", NULL);
exit(1);    }

    if (fork() == 0) {          /* Stage 2: pipe read +
output redirection */          dup2(pipefd[0],
STDIN_FILENO);          close(pipefd[0]);
close(pipefd[1]);

    int out_fd = open("filtered.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
dup2(out_fd, STDOUT_FILENO);          close(out_fd);          execlp("sort",
"sort", NULL);          exit(1);    }

    close(pipefd[0]); close(pipefd[1]);
wait(NULL); wait(NULL);

    printf("Execution Plan Verified. Resulting 'filtered.txt':\n");
system("cat filtered.txt");    return 0; } Output:

```

```

charanteja_a@charan:~$ nano exp10s2.c
charanteja_a@charan:~$ gcc exp10s2.c -o exp10s2
charanteja_a@charan:~$ ./exp10s2
[Task 2] Complex Command: 'grep apple < raw_metrics.txt | sort > filtered.txt'
Execution Plan Verified. Resulting 'filtered.txt':
apple 100
apple 300
charanteja_a@charan:~$ |

```

Task 3

To Detect Background Jobs, Launch Non-Blocking Processes, Return Prompt Immediately, Store Job Information, Monitor Execution, Test Background Tasks.

Implementation / work:

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

int is_background_command(char *cmd) {    size_t len = strlen(cmd);
while (len > 0 && (cmd[len - 1] == ' ' || cmd[len - 1] == '\n')) {
cmd[--len] = '\0';    }    if (len > 0 && cmd[len - 1] == '&') {
cmd[len - 1] = '\0';        return 1;    }    return 0; }

int main() {    char cmd[] = "sleep 1 &";
printf("Evaluating Input Command: '%s'\n", cmd);

    int bg = is_background_command(cmd);    if (bg) {
printf("[Task 3] Background operator '&' detected!\n");
pid_t pid = fork();    if (pid == 0) {
execlp("sleep", "sleep", "1", NULL);        exit(1);
}    printf("[Job 1] PID: %d launched in background.\n",
pid);    printf("Prompt returned immediately: teja27@TEJA:~$
\n");
}

```

Skill objective:

```

/* Parent continues immediately without blocking */
usleep(1200000); // Wait for sleep to conclude      int
status;        if (waitpid(pid, &status, WNOHANG) > 0) {
printf("[Job 1]+ Done\tsleep 1\n");                }
return 0; } Output:

```

```

charanteja_a@charan:~$ nano exp10s3.c
charanteja_a@charan:~$ gcc exp10s3.c -o exp10s3
charanteja_a@charan:~$ ./exp10s3
Evaluating Input Command: 'sleep 1 &'
[Task 3] Background operator '&' detected!
[Job 1] PID: 1558 launched in background.
Prompt returned immediately: teja27@TEJA:~$
[Job 1]+ Done    sleep 1
charanteja_a@charan:~$ |

```

Task 4

To Create Job Table, Store Process Groups, Track States, Update Status, Remove Completed Jobs, Validate Job Records.

Implementation / work:

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_JOBS 10

typedef enum { JOB_RUNNING, JOB_STOPPED, JOB_DONE } JobState;

typedef struct {
int job_id;      pid_t
pid;            JobState
state;          char
command[64]; } Job;

Job job_table[MAX_JOBS];
int job_count = 0;

void add_job(pid_t pid, const char *cmd) {
    if (job_count < MAX_JOBS) {
        job_table[job_count].job_id = job_count + 1;
        job_table[job_count].pid = pid;
        job_table[job_count].state = JOB_RUNNING;
        strncpy(job_table[job_count].command, cmd, 63);
        printf("Added to Job Table: [%d] PID: %d State: RUNNING Command: '%s'\n",
            job_table[job_count].job_id, pid, cmd);
        job_count++;
    }
}

void print_job_table() {
    printf("\n--- Active Shell Job Table ---\n");
    for (int i = 0; i < job_count; i++) {
        const char *st_str = (job_table[i].state ==
            JOB_RUNNING) ? "Running" :
            (job_table[i].state ==
            JOB_STOPPED) ? "Stopped" : "Done";
        printf("[%d] PID: %d State: %-8s Command: '%s'\n",
            job_table[i].job_id, job_table[i].pid, st_str,
            job_table[i].command);
    }
}

int main() {
    add_job(1045, "python3 worker_node.py &");
    add_job(1046, "ffmpeg -i stream.mp4 out.avi &");
    add_job(1047, "make -j4 build_project &");
}

```

Skill objective:

```
print_job_table();    printf("\nSimulating state change: Job [2]
suspended via SIGTSTP...\n");    job_table[1].state = JOB_STOPPED;
print_job_table();    return 0; }
```

Output:

```
charanteja_a@charan:~$ nano exp10s4.c
charanteja_a@charan:~$ gcc exp10s4.c -o exp10s4
charanteja_a@charan:~$ ./exp10s4
Added to Job Table: [1] PID: 1045 State: RUNNING Command: 'python3 worker_node.py &'
Added to Job Table: [2] PID: 1046 State: RUNNING Command: 'ffmpeg -i stream.mp4 out.avi &'
Added to Job Table: [3] PID: 1047 State: RUNNING Command: 'make -j4 build_project &'

--- Active Shell Job Table ---
[1] PID: 1045 State: Running  Command: python3 worker_node.py &
[2] PID: 1046 State: Running  Command: ffmpeg -i stream.mp4 out.avi &
[3] PID: 1047 State: Running  Command: make -j4 build_project &

Simulating state change: Job [2] suspended via SIGTSTP...

--- Active Shell Job Table ---
[1] PID: 1045 State: Running  Command: python3 worker_node.py &
[2] PID: 1046 State: Stopped  Command: ffmpeg -i stream.mp4 out.avi &
[3] PID: 1047 State: Running  Command: make -j4 build_project &
charanteja_a@charan:~$ |
```